

Algorithms & Complexity

CS203.3

Lecture 1: iterative and recursive algorithms

M.G.Noel A.S Fernando (PhD)

Professor

NSBM/Dept. of Information Systems Engineering, UCSC



What is an algorithm?

- Define ?

What is an algorithm?

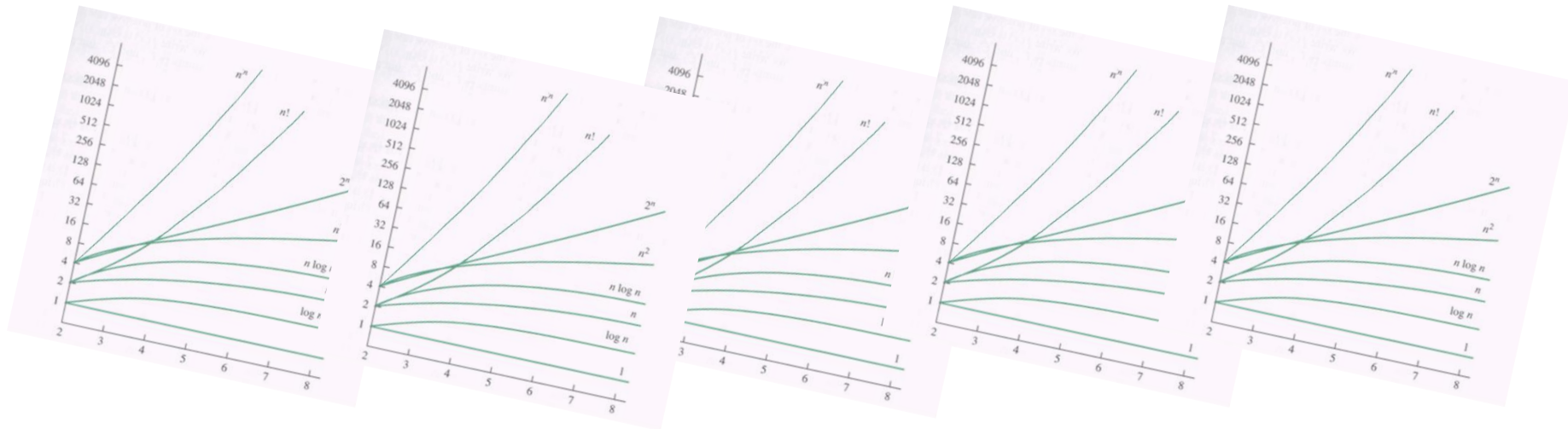
- Definition: Algorithm

Computer algorithm is

- **A detailed step by step method for solving a problem**
- **A sequence of clear-cut instructions for solving the problem in a finite amount of time.**
- **Any well-defined computational procedure that takes collection of elements as input and produces collection of elements as output.**
- **A set of rules for carrying out calculation either by hand or on a machine.**
- **An abstraction of a program to be executed on a physical machine (model of computation).**

Purpose

- To estimate how long a program will run.
- To estimate the largest input that can reasonably be given to the program.
- To compare the efficiency of different algorithms.
- To help focus on the parts of code that are executed the largest number of times.
- To choose an algorithm for an application.



Write a pseudocode algorithm calculate factorial

- Iterative algorithm
- Recursive algorithm

Recursive and Iterative Algorithms

- Iterative and recursive algorithms are two techniques used for problem solving.
- Iterative algorithms use loops to repeat steps, while recursive algorithms use function calls where a function calls itself.
- Recursive algorithms are generally slower but use less memory, while iterative algorithms are faster but use more memory.
- Example

```
//Recursive Impementation
int fact(int n)
{
    if (n == 0)
        return 1
    return n * fact(n - 1)
}
```

```
//Iterative Impementation
int fact(int n)
{
    int output = 1
    for (int i = 2; i <= n; i = i + 1)
        output = output*i
    return output
}
```

Pseudo Code - **nth factorial**

Overview of Recursive and Iterative Algorithms

- Iteration and recursion are key computer program writing techniques used in creating algorithms and developing the software.
- Both are based on control structures and mainly different used different problem-solving approaches.

Iterative	Recursive
○ Uses looping statements such as for loop, while loop or do-while loop to repeat the same steps.	○ Uses a function calls itself again and again till the base condition (stopping condition) is satisfied.
○ Uses the repetition structure	○ Uses the selection structure
○ An infinite loop occurs with iteration if the loop-condition test never becomes false.	○ Infinite recursion occurs if the recursion step does not reduce the problem in a manner that converges on some condition. (base case)
○ Faster operation as uses the permanent storage area	○ Due to overhead of maintaining stack the operation is slower

Overview of Recursive and Iterative Algorithms (Cont..)

Iterative	Recursive
○ Consume less memory	○ uses exorbitant memory
○ Termination occurs when the loop condition fails	○ Stops when the base condition is fulfilled
○ Infinite looping uses CPU cycles repeatedly	○ Infinite recursion can crash the system
○ Makes tangling code	○ Makes code smaller
○ Rare choice for complex problem	○ Best choice for complex problem solving
○ Finding average of a data series, creating multiplication table are the basic applications	○ Applications including, Factorial , Fibonacci Series

Overview of Recursive and Iterative Algorithms (Cont..)

Iterative	Recursive
<pre>int main() { int i, n = 5, fact = 1; for(i = 1; i <= n; ++i) fact = fact * i; printf("Factorial for 5 is %d", fact); getch(); }</pre>	<pre>int fact(int n){ if(n == 0) return 1; else return n * factorial(n-1); } int main() { printf("Factorial for 5 is %d", fact(5)); getch(); }</pre>

Why learn recursion?

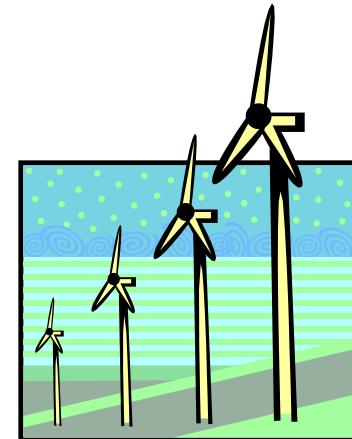
- cultural experience - A different way of thinking of problems
- Can solve some kinds of problems better than iteration
- Leads to elegant, simplistic, short code (when used well)
- Many programming languages ("functional" languages such as Scheme, ML, and Haskell) use recursion exclusively (no loops)

Recursion

- When a function (or procedure) calls itself, Such a function is called "recursive".
- If the call is via one or more other functions, then this group of functions are called "mutually recursive".
- for example, Function A calls Function B, which calls Function C, which then calls Function A again , this is called **mutual recursion**.
- It means the functions depend on each other to complete their tasks recursively.

Basic Recursions

- Break a problem into smaller identical problems
 - Each recursive call solves an identical but smaller problem.
- Stop the break-down process at a special case whose solution is obvious, (termed **a base case**)
 - Each recursive call tests the base case to eventually stop.
 - Otherwise, we fall into an infinite recursion.



Getting downstairs



- Need to know two things:
 - Getting down one stair
 - Recognizing the bottom

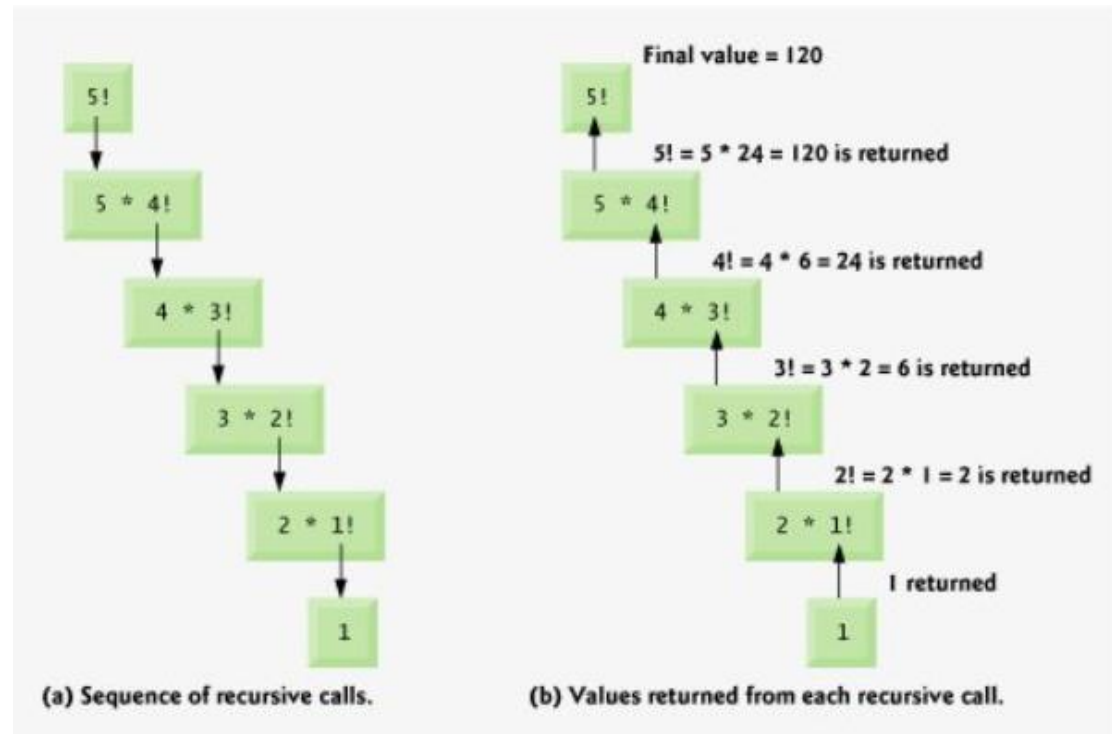
- Most code will look like:

```
if (simplest case) {  
    compute and return solution  
} else {  
    divide into similar subproblem(s)  
    solve each subproblem recursively  
    assemble the overall solution  
}
```

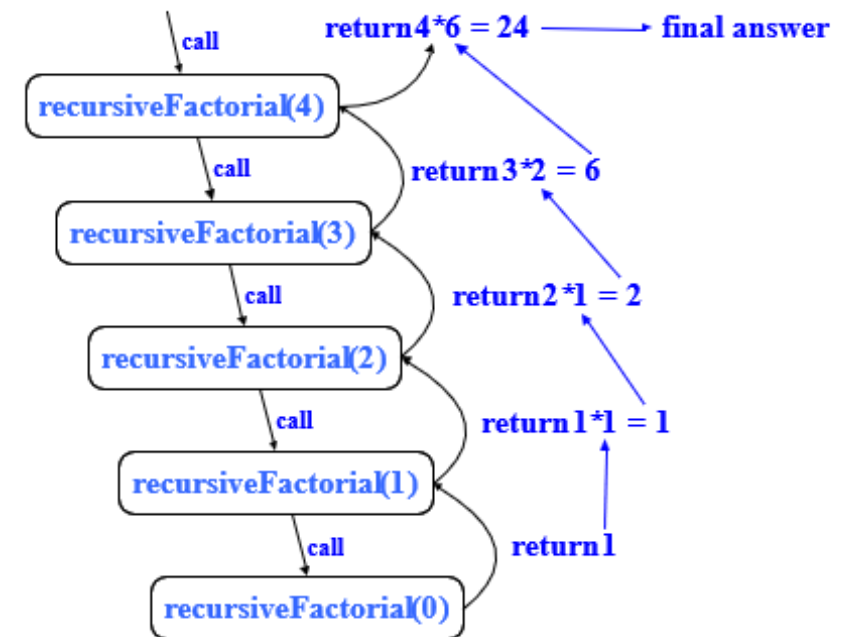
Recursive Definitions

- Recursion
 - Process of solving a problem by reducing it to smaller versions of itself
- Recursive definition
 - Definition in which a problem is expressed in terms of a smaller version of itself
 - Has one or more base cases

Recursive example



Example recursion trace:



Write recursive definition for factorial

Example 1: recursive definition for factorial

- Definition:

$\text{factorial}(n) = n * (n-1) * (n-2) * \dots * 1$ for any integer $n > 0$

$\text{factorial}(1) = 1$

- Recursive relation;

$\text{factorial}(n) = n * [(n-1) * (n-2) * \dots * 1]$

$= n * \text{factorial}(n-1)$

Looks so nice, but how about $n = 1$?

$\text{factorial}(1) = 1 * \text{factorial}(0)$ but the argument to $\text{factorial}()$ must be positive

- Final recursive definition:

$\text{factorial}(n) = 1$ if $n = 1$

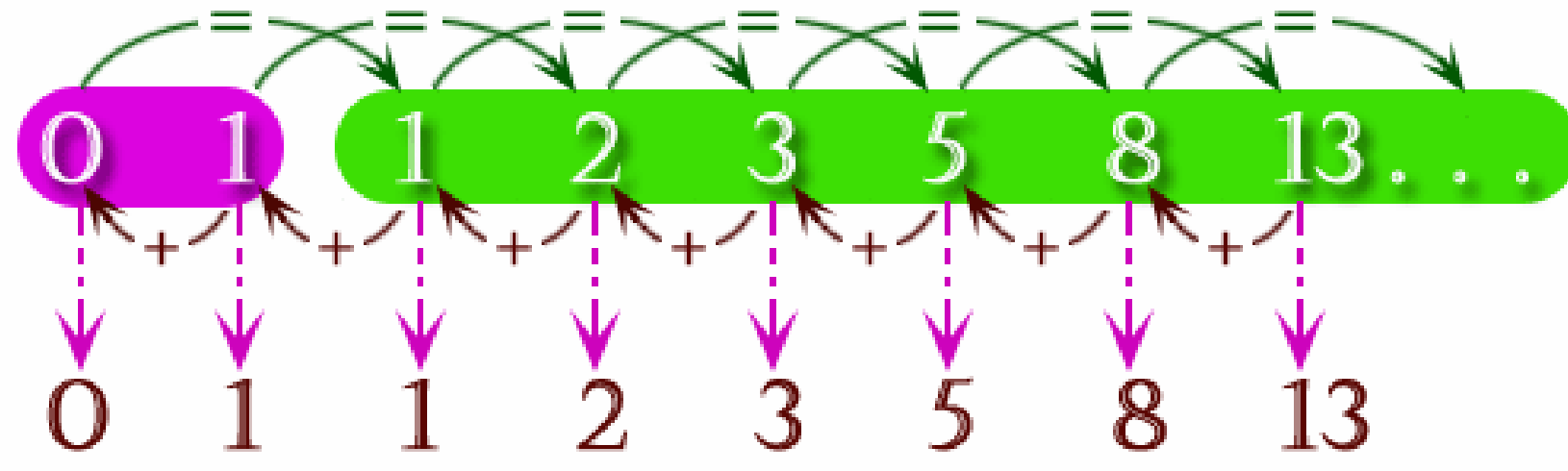
$= n * \text{factorial}(n-1)$ if $n > 1$

What is the Fibonacci sequence?

- The Fibonacci sequence is a set of integers (the Fibonacci numbers) that starts with a zero.
- The sequence follows the rule that each number is equal to the sum of the preceding two numbers.
- Two base cases 0 and 1 (first two numbers)
- Fibonacci sequence /series
- 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233 ...
- Drive an equation to form a Fibonacci sequence /series
- ????

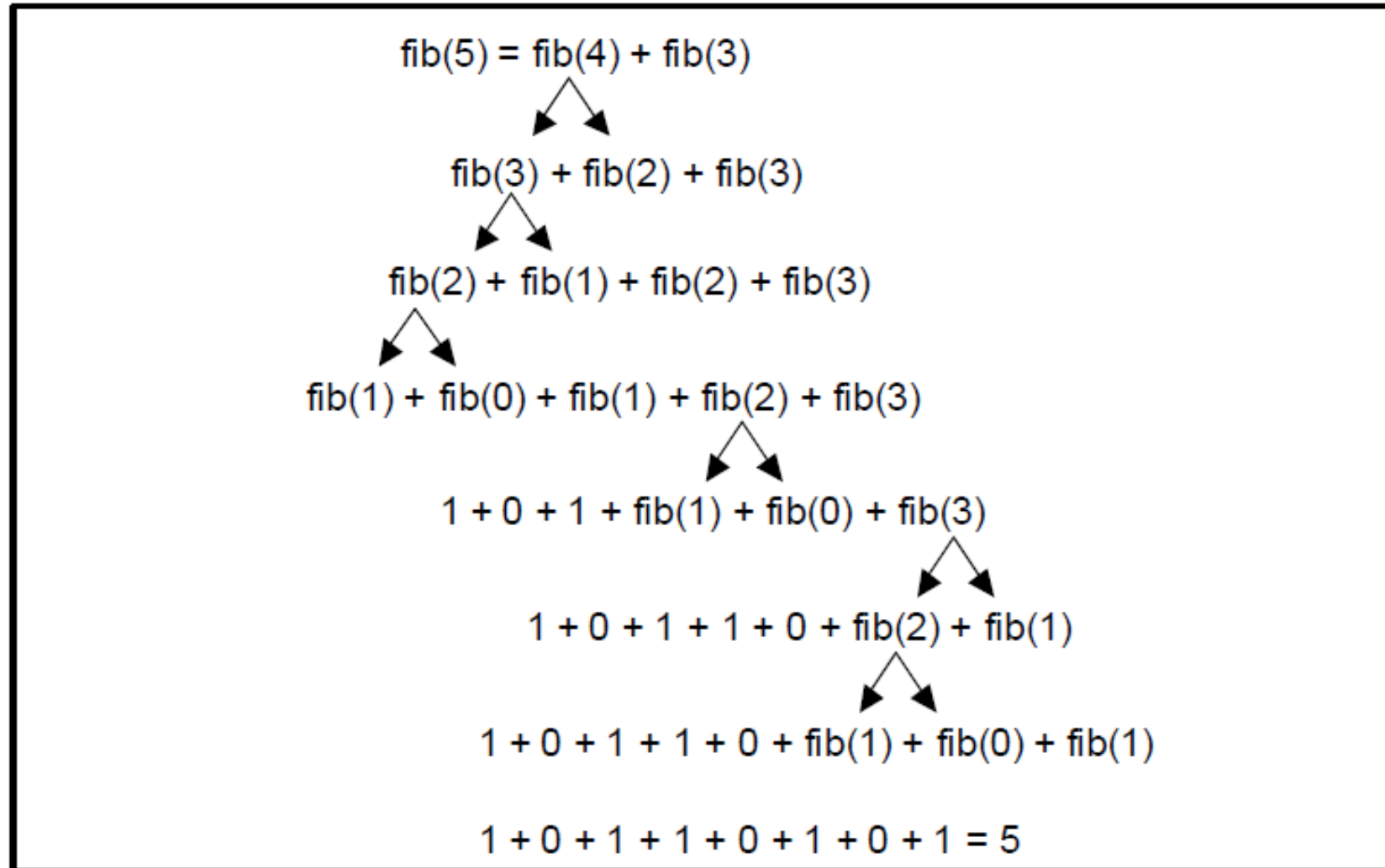
What is the Fibonacci sequence?

Fibonacci Sequence :



How to Compute $\text{fib}(5)$

How to Compute fib(5)



What is the Fibonacci sequence?

- The Fibonacci sequence is a set of integers (the Fibonacci numbers) that starts with a zero.
- The sequence follows the rule that each number is equal to the sum of the preceding two numbers.
- Two base cases 0 and 1 (first two numbers)
- Fibonacci sequence /series
- 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233 ...
- Drive an equation to form a Fibonacci sequence /series
- $F_0 = 0$
- $F_1 = 1$
- $F_n = F_{n-1} + F_{n-2}$ (applies to all other integers)

Why Are Fibonacci Numbers Important in Nature?

A Sample video

- <https://www.youtube.com/watch?v=-GVQ2-3Nv2s>

Write a pseudocode algorithm or C code to generate Fibonacci numbers in iterative manner.

Write a pseudocode algorithm or C code to generate Fibonacci numbers in iterative manner.

```
#include <stdio.h>
int main() {
    int i, n;
    // initialize first and second terms
    int t1 = 0, t2 = 1;
    // initialize the next term (3rd term)
    int nextTerm = t1 + t2;
    // get no. of terms from user
    printf("Enter the number of terms: ");
    scanf("%d", &n);
```

```
    // print the first two terms t1 and t2
    printf("Fibonacci Series: %d, %d, ", t1, t2);
    // print 3rd to nth terms
    for (i = 3; i <= n; ++i) {
        printf("%d, ", nextTerm);
        t1 = t2;
        t2 = nextTerm;
        nextTerm = t1 + t2;
    }
    return 0;
}
```

Write recursive definition for Fibonacci numbers

Recursive definition for Fibonacci numbers

- Another well known recursive sequence is the [Fibonacci numbers](#).
The first few elements of this sequence are: 0, 1, 1, 2, 3, 5, 8, 13, 21...

$$\text{fib}(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ \text{fib}(n-1) + \text{fib}(n-2) & \text{if } n \geq 2 \end{cases}$$

Write a Pseudocode algorithm to generate Fibonacci numbers?

Write a Pseudocode algorithm to generate a Fibonacci number?

- **function** fib is:

input: integer n such that $n \geq 0$

1. if n is 0, **return** 0
 2. if n is 1, **return** 1
 3. otherwise, **return** [fib(n-1) + fib(n-2)]
- end** fib

Write a C code to compute a given Fibonacci number.

Write a C code to compute a given Fibonacci number.

```
#include <stdio.h>
// Function to calculate the nth Fibonacci
number using recursion
int nthFibonacci(int n){
    // Base case: if n is 0 or 1, return n
    if (n <= 1){
        return n;
    }
    // Recursive case: sum of the two
preceding Fibonacci numbers
    return nthFibonacci(n - 1) +
nthFibonacci(n - 2);
}
```

```
int main(){
    int n = 5;
    int result = nthFibonacci(n);
    printf("%d\n", result);
    return 0;
}
```

Python code (Multiple Languages)

```
def fib(n):  
    if n==0:  
        return 0  
    elif n==1:  
        return 1  
    else:  
        return fib(n-1)+fib(n-2)  
for i in range(10):  
    print(fib(i))
```


Write a C program to find Fibonacci numbers up to n

Write a C program to find Fibonacci numbers up to n

Write a C program to find Fibonacci numbers up to n

```
int fibonacci(int n) {  
    if (n==1)  
        return 0;  
    else if (n==2)  
        return 1;  
    else  
        return fibonacci(n-1) + fibonacci(n-2);  
} //fibonacci
```

```
int fibonacciPrint(int n) {  
    for(int i=1; i<=n; i++)  
        printf("%d, ", fibonacci(i));  
} //fibonacciPrint
```

What is infinite Recursion?

Infinite Recursion

- If a recursion never reaches a base case, it goes on making recursive calls forever, and the program never terminates.
- This is known as infinite recursion, and it is generally not a good idea.
- As the base condition is never met, the recursion carries on infinitely.

What is a "Common Factor" ?

Let us say you have worked out the factors of two numbers:

Example: Factors of 12 and 30

Factors of 12 are 1, 2, 3, 4, 6 and 12

Factors of 30 are 1, 2, 3, 5, 6, 10, 15 and 30

Then the common factors are those that are found in both lists:

- Notice that 1, 2, 3 and 6 appear in both lists?
- So, the common factors of 12 and 30 are: 1, 2, 3 and 6

Greatest Common Factor

The highest number that divides exactly into two or more numbers.

It is the "greatest" thing for simplifying fractions!

Let's start with an Example ...

Greatest Common Factor
of 12 and 16

- Find all the **Factors** of each number,
 - Circle the **Common** factors,
- Choose the **Greatest** of those

The "Greatest Common Factor" is the largest of the common factors (of two or more numbers)



Exercise

- Write a recursive program to determine the Greatest Common Factor

Recursive function for gcd

```
def gcd(a,b):  
    t = b  
    b = a % b  
  
    if b == 0:  
        return t  
    else:  
        return gcd(t,b)  
  
print(gcd(12,6))
```

Greatest Common Divisor

- Write a simple **recursive C program** to find the **GCD (Greatest Common Divisor)** of two numbers using **Euclid's algorithm**:

Greatest Common Divisor

Write a simple recursive C program to find the GCD (Greatest Common Divisor) of two numbers using Euclid's algorithm:

```
#include <stdio.h>
// Function to find GCD using recursion
int gcd(int a, int b) {
    if (b == 0)
        return a;
    else
        return gcd(b, a % b);
}
```

```
int main() {
    int num1, num2;

    printf("Enter two numbers: ");
    scanf("%d %d", &num1, &num2);

    printf("GCD of %d and %d is %d\n", num1,
num2, gcd(num1, num2));

    return 0;
}
```

Exercise 1

- **Write a Program to find sum of first n natural numbers**
- ***Input: $n = 3$***
Output: 6
Explanation: $1 + 2 + 3 = 6$
- ***Input: $n = 5$***
Output: 15
Explanation: $1 + 2 + 3 + 4 + 5 = 15$
- Can you develop a code in iterative manner?

Program to find sum of first n natural numbers

Input: e.g n = 5

```
#include <stdio.h>
int findSum(int n){
    int sum = 0;
    // iterating over all the numbers
    // between 1 to n
    for (int x = 1; x <= n; x++)
        sum = sum + x;
    return sum;
}
```

```
int main()
{
    int n = 5;
    printf("%d", findSum(n));
    return 0;
}
```

Write a recursive definition to get The Sum of the First N Positive Integers

Write a recursive definition to get The Sum of the First N Positive Integers

- Definition:

$$\text{sum}(n) = n + (n-1) + (n-2) + \dots + 1 \text{ for any integer } n > 0$$

- Recursive relation;

$$\begin{aligned}\text{sum}(n) &= n + [(n-1) + (n-2) + \dots + 1] \\ &= n + \text{sum}(n-1)\end{aligned}$$

Looks so nice, but how about $n == 1$?

$\text{sum}(1) = 1 + \text{sum}(0)$, but the argument to $\text{sum}()$ must be positive

- Final recursive definition:

$$\begin{aligned}\text{sum}(n) &= 1 && \text{if } n = 1 \text{ (Base case)} \\ &= n + \text{sum}(n-1) && \text{if } n > 1 \text{ (Recursive call)}\end{aligned}$$

To get The Sum of the First N Positive Integers in a recursive way

- **Approach**
- Given a number n ,
- To calculate the sum, we will use a recursive function $\text{recSum}(n)$.
- **BaseCondition:** If $n \leq 1$ then $\text{recSum}(n)$ returns the n .
- **Recursive call:** return $n + \text{recSum}(n-1)$.

Write a C code to get The Sum of the First N Positive Integers in a recursive way

Write a C code to get The Sum of the First N Positive Integers in a recursive way

```
// C program to find the sum of n
// natural numbers using recursion
#include <stdio.h>
// Returns the sum of first n
// natural numbers
int recSum(int n)
{
    // Base condition
    if (n <= 1)
        return n;
```

```
// Recursive call
    return n + recSum(n - 1);
}
// Driver code
int main()
{
    int n = 10;
    printf("Sum = %d ", recSum(n));
    return 0;
}
```

Exercise

- **Write a recursive definition for multiples of 3.**

Exercise -Write a recursive definition for multiples of 3.

- Think of a recursive version of the function $f(n) = 3 * n$,
- (i.e. the multiples of 3)
- $n=1, f(1)=3$
- $n=2, f(2)=6$
- $n=3, f(n)=9$
- $n=10, f(n)=30$
- $n=N, f(n)=3*N$
- Write a recursive definition for multiples of 3?

Write a recursive definition for multiples of 3

Mathematically, we can write it like this:

$$f(1)=3;$$

$$f(n)=f(n-1)+3$$

Write a pseudocode algorithm to generate multiples of 3.

Write a pseudocode algorithm to generate multiples of 3.

A pseudocode can be written like this :

```
def mult3(n):  
    if n == 1:  
        return 3  
    else:  
        return mult3(n-1) + 3
```

```
for i in range(1,10):  
    print(mult3(i))
```

Binary Search algorithm

- **Binary Search** is a search algorithm that is used to find the position of an element (target value) in a sorted array.
- The **array** should be sorted prior to applying a binary search.
- Working of Binary Search
- The **binary search algorithm** works by comparing the element to be searched by the middle element of the array and based on this comparison follows the required procedure.
 - **Case 1 : element = middle**, the element is found return the index.
 - **Case 2 : element > middle**, search for the element in the sub-array starting from **middle+1** index to **n**.
 - **Case 3 : element < middle**, search for element in the sub-array starting from 0 index to **middle -1**.

Binary Search algorithm

- Algorithm
- **Parameters** `initial_value` , `end_value`

Step 1 : Find the middle element of array. using ,
$$\text{middle} = \text{initial_value} + \text{end_value} / 2 ;$$

Step 2 : If `middle` = element, return 'element found' and index.

Step 3 : if `middle` > element, call the function with `end_value` = `middle` - 1 .

Step 4 : if `middle` < element, call the function with `start_value` = `middle` + 1 .

Step 5 : exit.

The implementation of the binary search algorithm function- Iterative

```
#include <stdio.h>
int iterativeBinarySearch(int array[], int start_index, int end_index, int element){
    while (start_index <= end_index){
        int middle = start_index + (end_index- start_index )/2;
        if (array[middle] == element)
            return middle;
        if (array[middle] < element)
            start_index = middle + 1;
        else
            end_index = middle - 1;
    }
    return -1;
}
```

```
int main(void){
    int array[] = {1, 4, 7, 9, 16, 56, 70};
    int n = 7;
    int element = 16;
    int found_index = iterativeBinarySearch(array, 0, n-1, element);
    if(found_index == -1 ) {
        printf("Element not found in the array ");
    }
    else {
        printf("Element found at index : %d",found_index);
    }
    return 0;
}
```

The implementation of the binary search algorithm function- Recursive

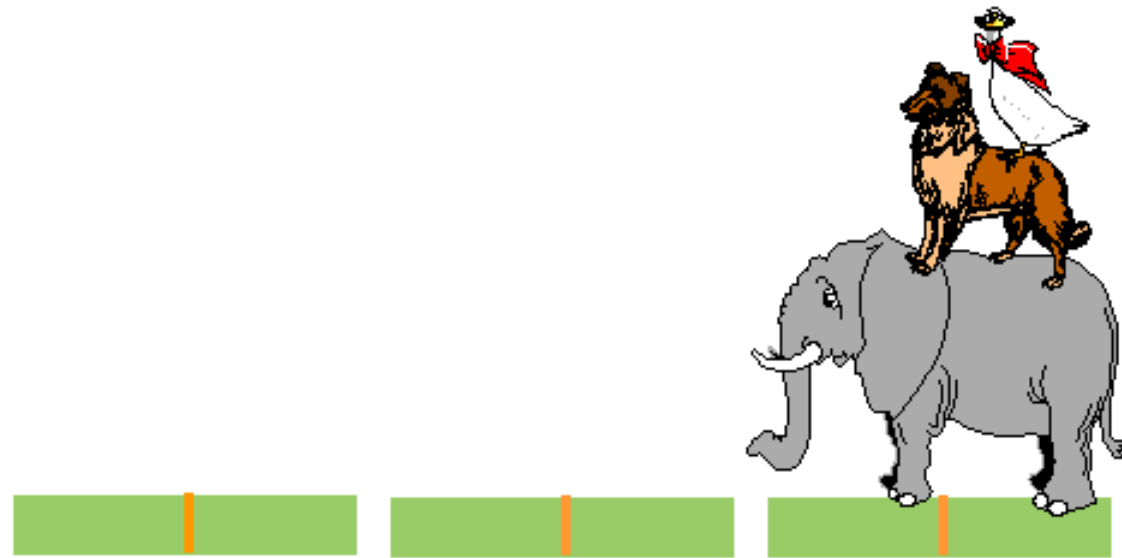
```
#include <stdio.h>

int recursiveBinarySearch(int array[], int start_index, int end_index, int element){
    if (end_index >= start_index){
        int middle = start_index + (end_index - start_index)/2;
        if (array[middle] == element)
            return middle;
        if (array[middle] > element)
            return recursiveBinarySearch(array, start_index, middle-1, element);
        return recursiveBinarySearch(array, middle+1, end_index, element);
    }
    return -1;
}
```

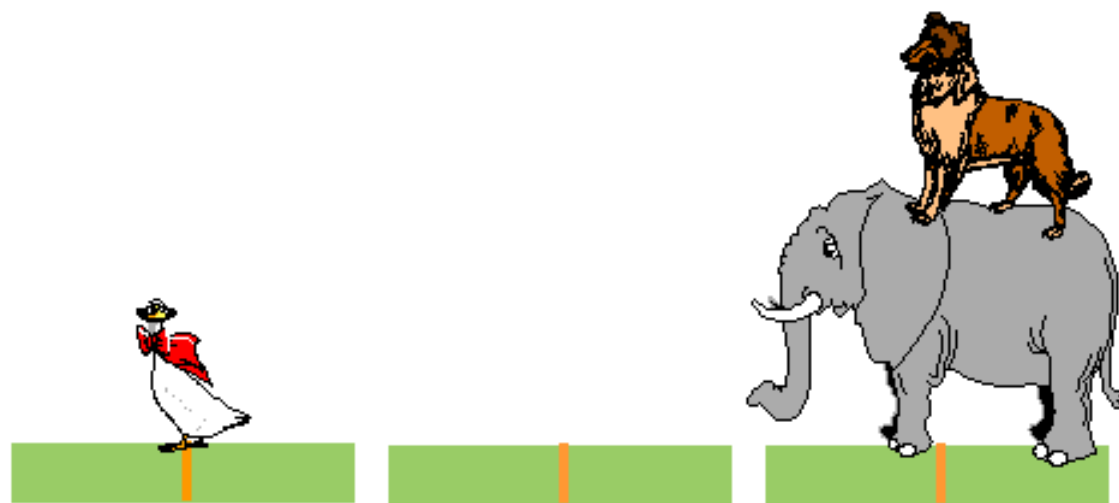
```
int main(void){
    int array[] = {1, 4, 7, 9, 16, 56, 70};
    int n = 7;
    int element = 9;
    int found_index = recursiveBinarySearch(array, 0, n-1, element);
    if(found_index == -1 ) {
        printf("Element not found in the array ");
    }
    else {
        printf("Element found at index : %d",found_index);
    }
    return 0;
}
```

Analogy for Towers of Hanoi

Analogy for Towers of Hanoi



Analogy for Towers of Hanoi



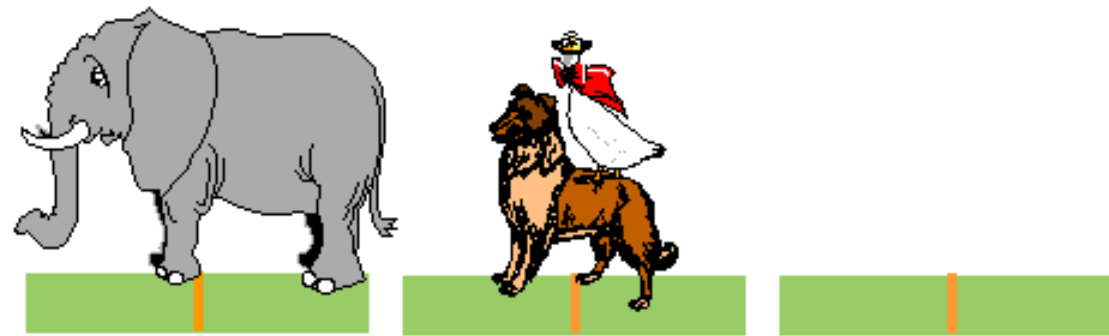
Analogy for Towers of Hanoi



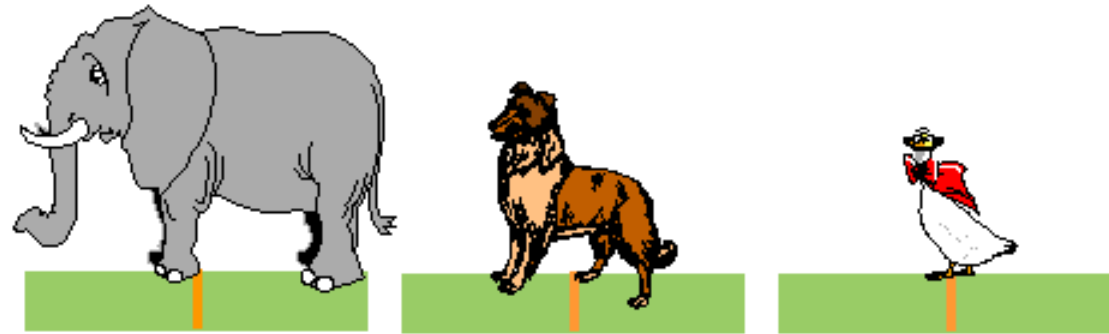
Analogy for Towers of Hanoi



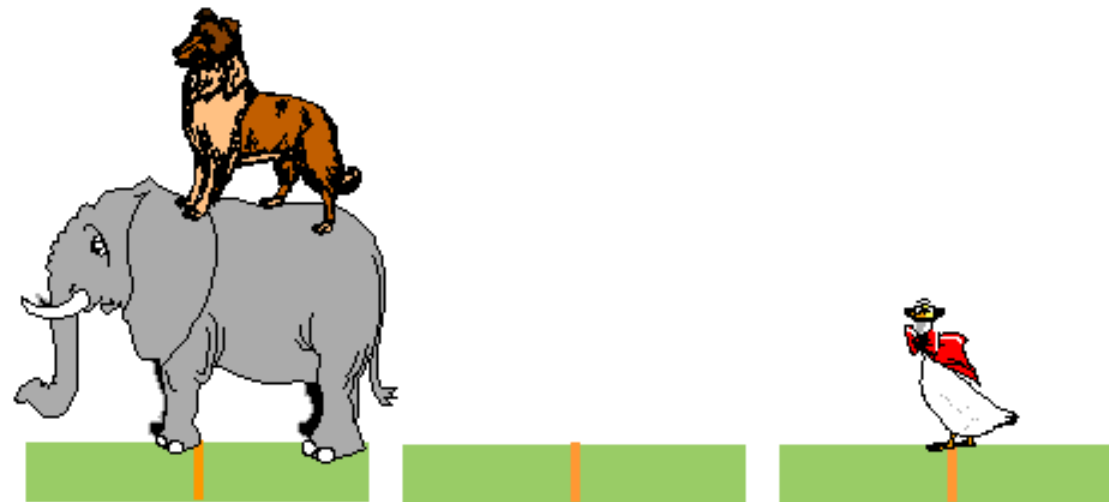
Analogy for Towers of Hanoi



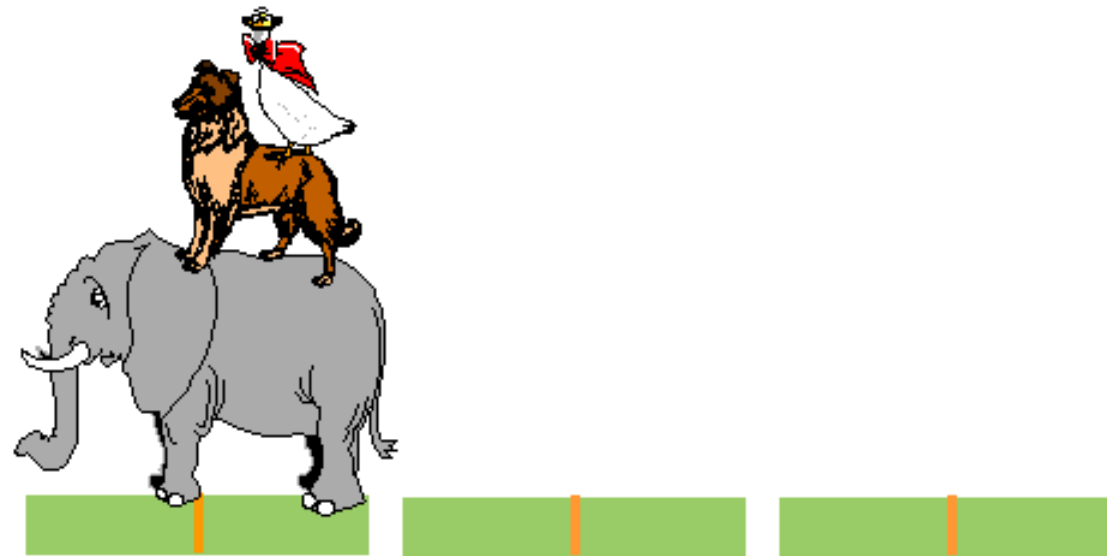
Analogy for Towers of Hanoi



Towers of Hanoi



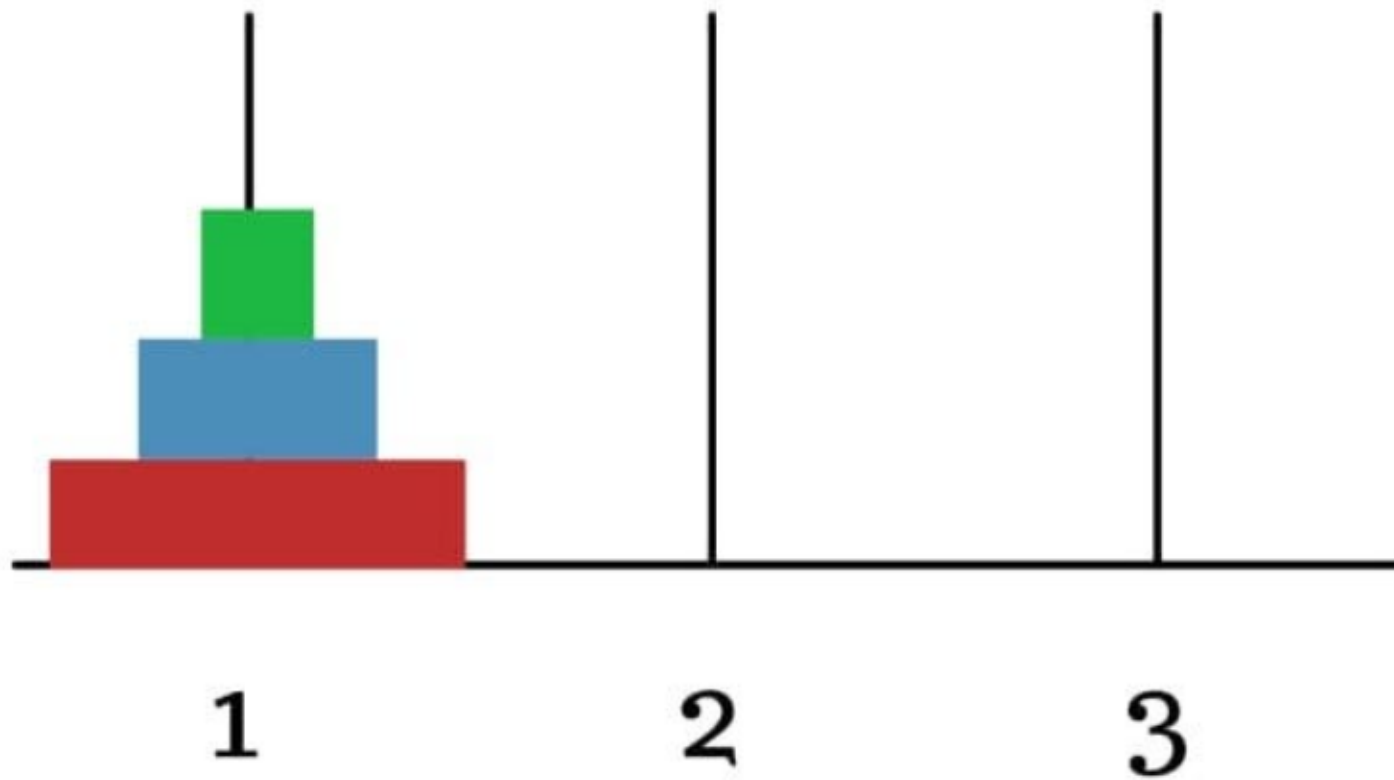
Analogy for Towers of Hanoi



Towers of Hanoi

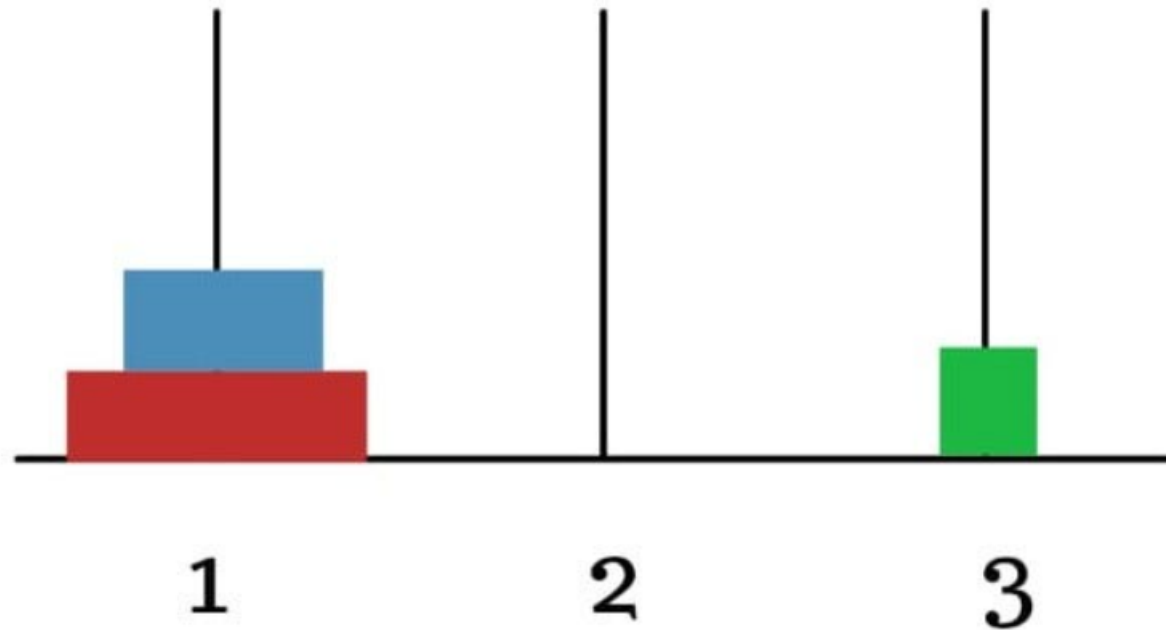
- **The logic behind solving the Tower of Hanoi for three disks :**
- **Objective :** To solve the Tower of Hanoi puzzle that contains three disks.
- The stack of disks has to be shifted from Rod 1 to Rod 3 by abiding to the set of rules that has been mentioned above.

Towers of Hanoi



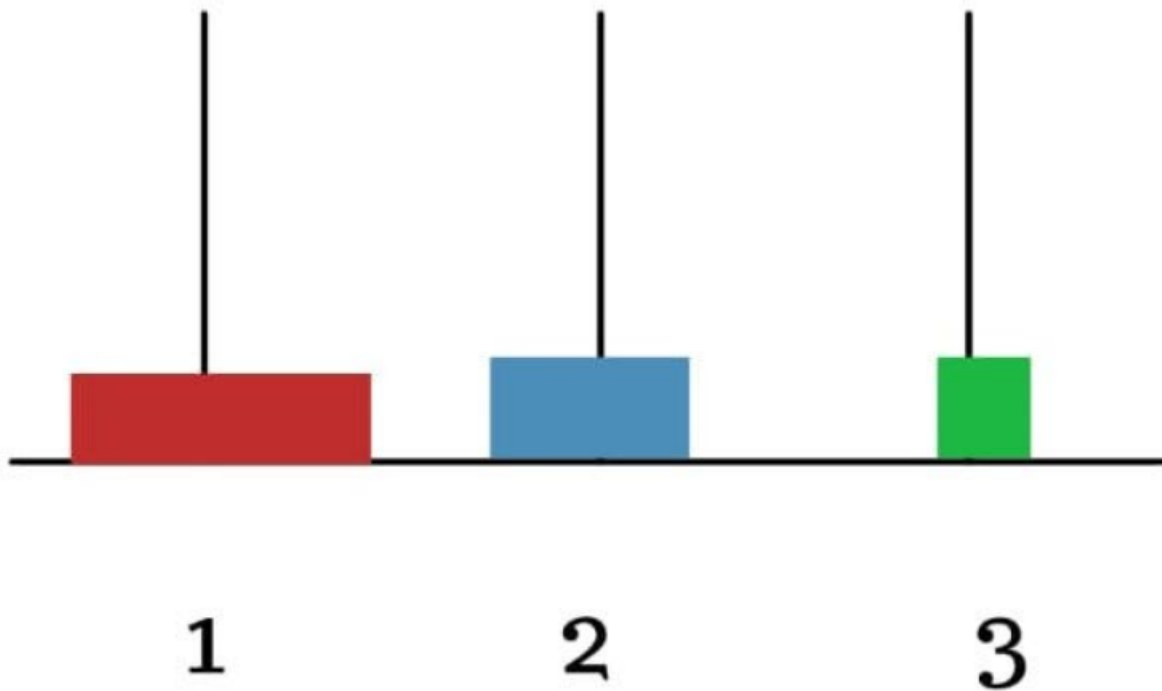
Towers of Hanoi

- **Step 1** : The smallest green disk , the uppermost disk on the stack is shifted from rod 1 to rod 3.



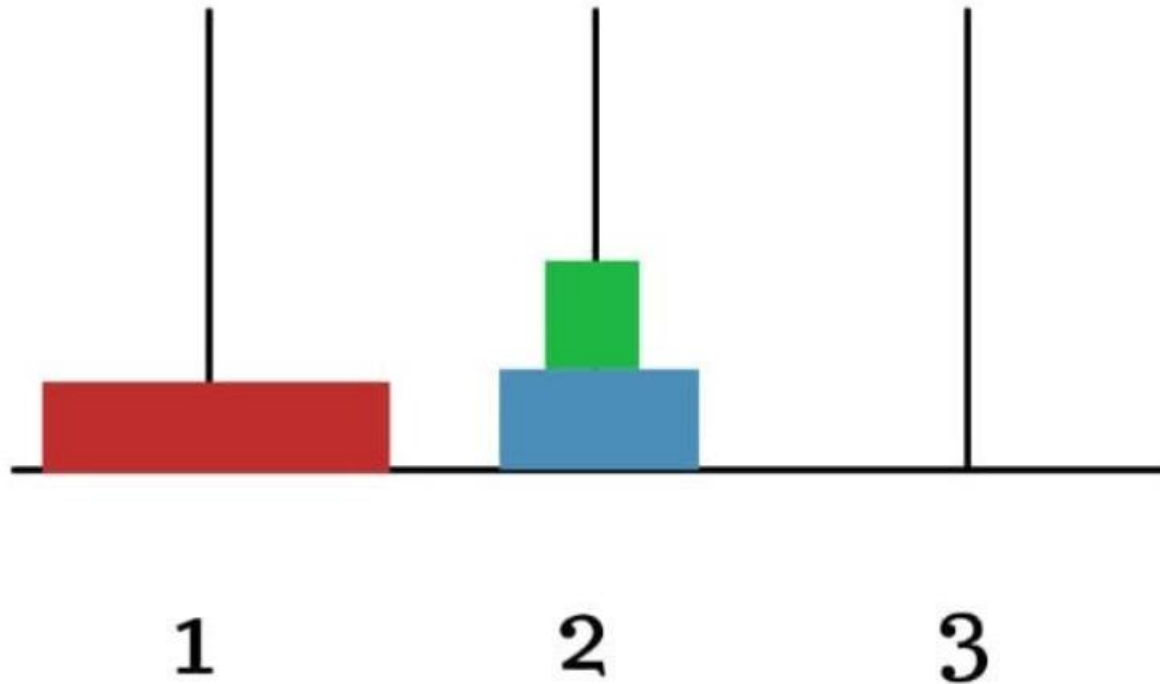
Towers of Hanoi

- **Step 2** : Next the uppermost disk on rod 1 is the blue colored disk which is shifted to rod 2.



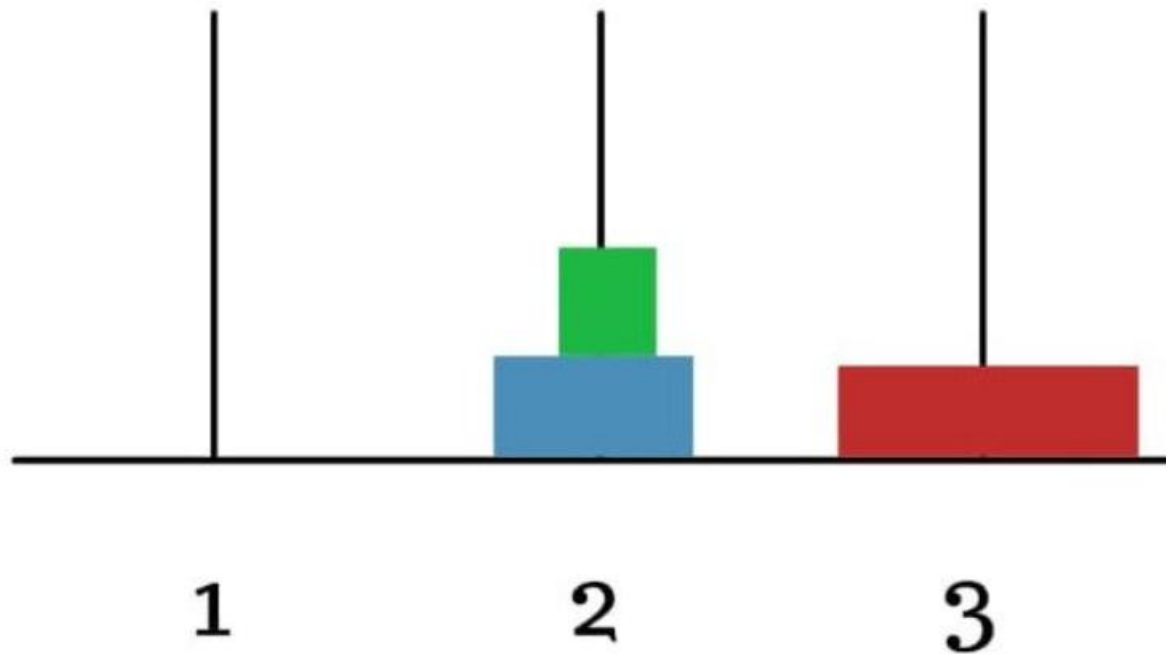
Towers of Hanoi

- **Step 3 :** The smallest disk placed on rod 3 is shifted back on to the top of rod 2.



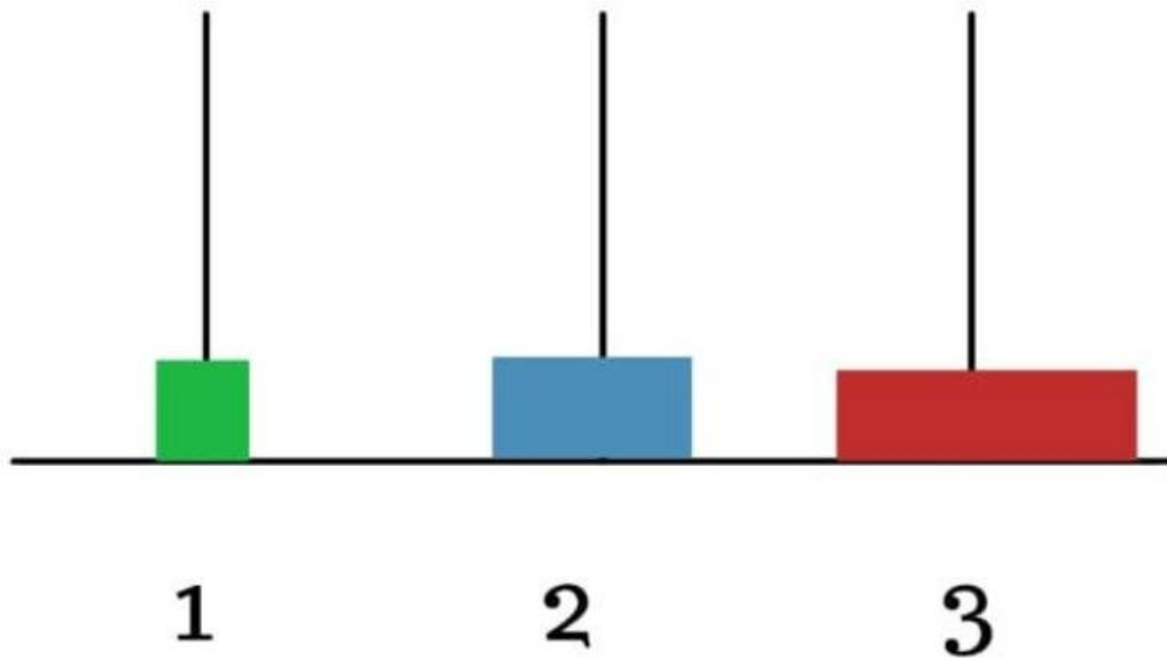
Towers of Hanoi

- **Step 4** : So now the largest red disk is allowed to be shifted from rod 1 to its destination rod 3.



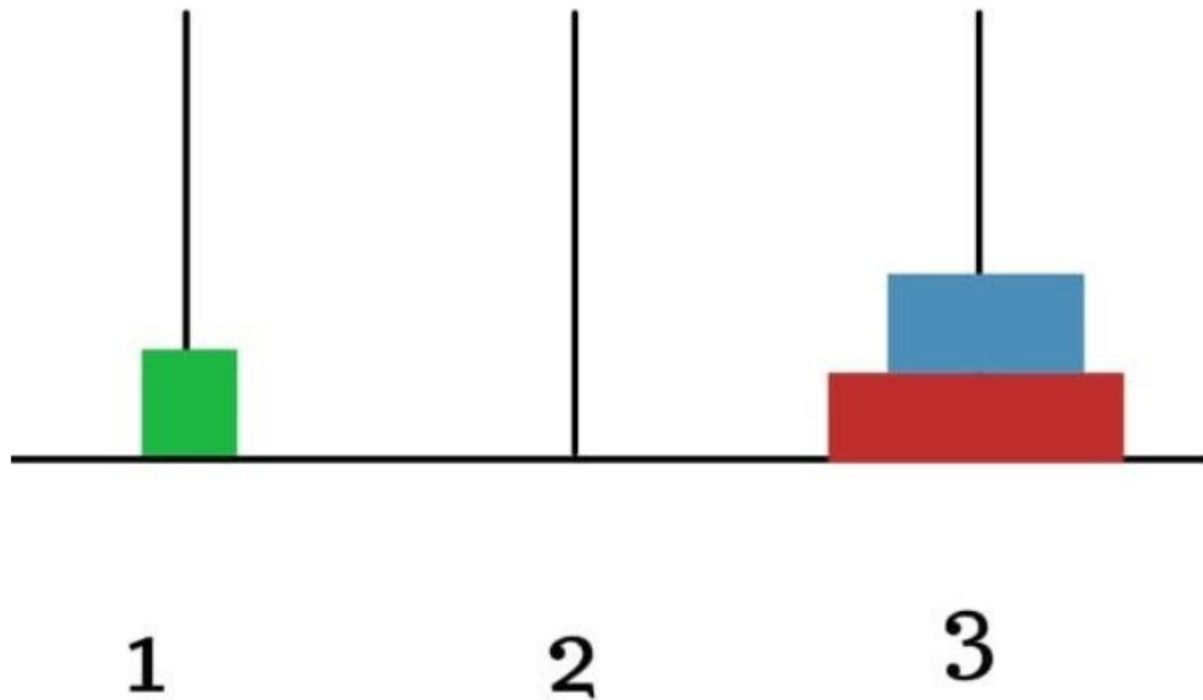
Towers of Hanoi

- **Step 5 :** Now the two disks on rod 2 has to be shifted to its destination rod 3 on top of the red disk , so first the smallest green disk on top of the blue rod is shifted to rod 1 .



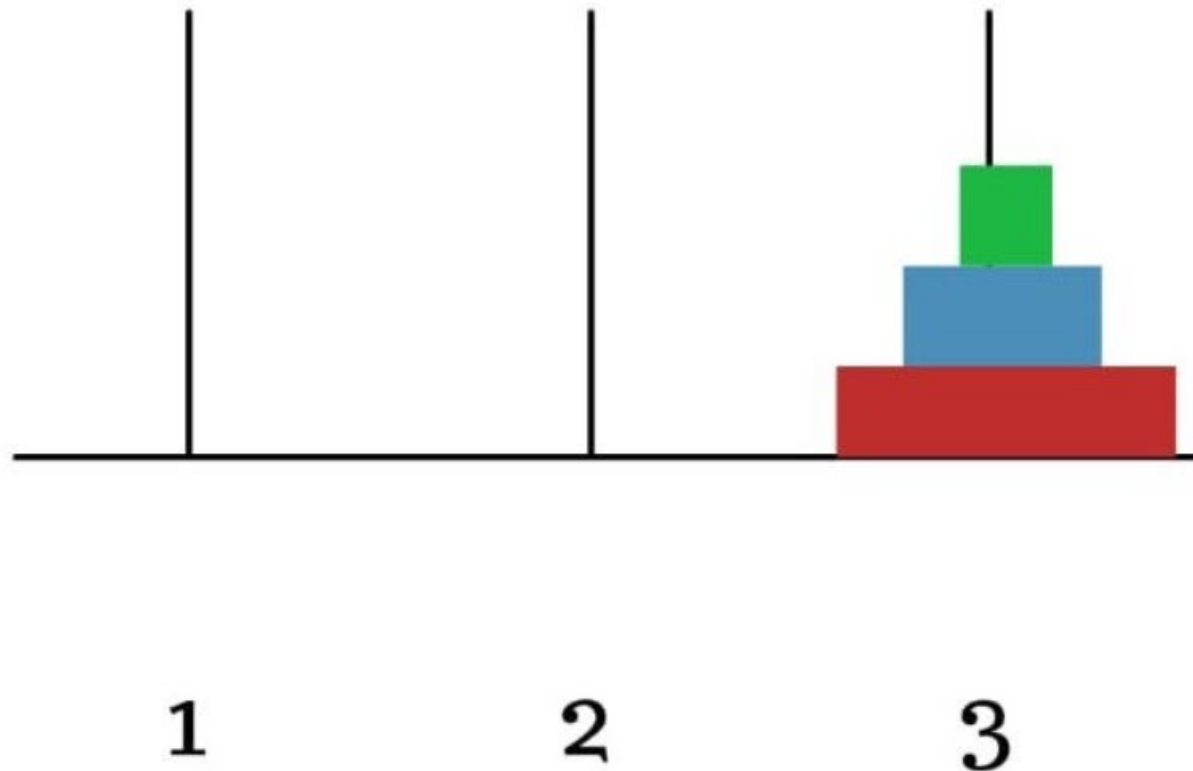
Towers of Hanoi

- **Step 6** : Next the blue disk is permitted to be shifted to its destination rod 3 that will stack on to the top of the red disk.



Towers of Hanoi

- **Step 7** : Finally , the smallest green colored rod is also shifted to rod 3 , which would now be the uppermost rod on the stack .

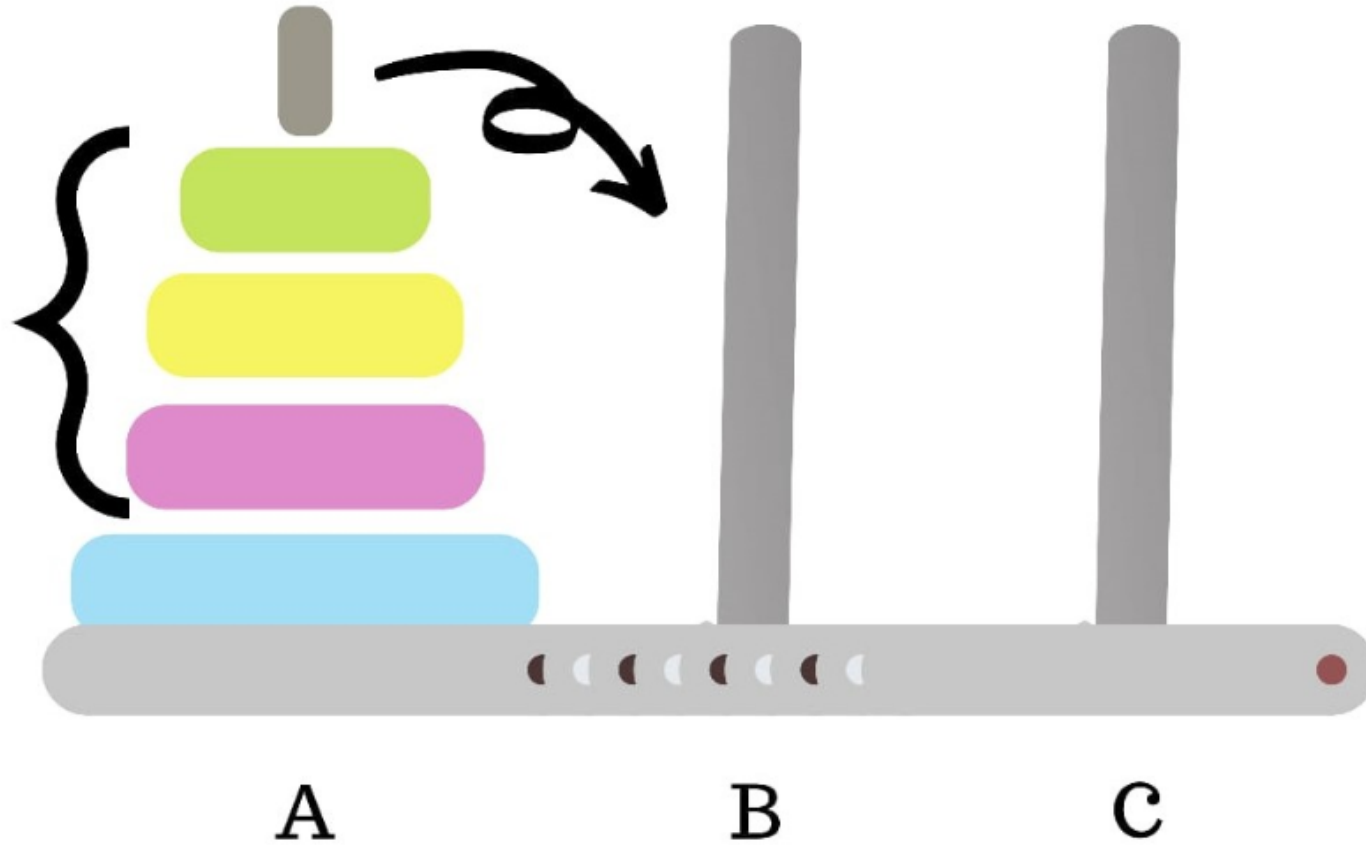


Recursive Algorithm

- To write an algorithm for Tower of Hanoi,
- first, we need to learn how to solve this problem with lesser number of disks, say $\rightarrow$ 1 or 2.
- We mark three towers with name, **source**, **destination** and **aux** (only to help moving the disks). If we have only one disk, then it can easily be moved from source to destination peg.
- If we have 2 disks –
- First, we move the smaller (top) disk to aux peg.
- Then, we move the larger (bottom) disk to destination peg.
- And finally, we move the smaller disk from aux to destination peg.
- See the animation
- https://www.tutorialspoint.com/data_structures_algorithms/images/tower_of_hanoi_two_disks.gif

Strategies :

1. Recursively solve the puzzle of shifting disks 1 , 2 , 3 from Rod A to Rod B.



Strategies :

2.Then move the largest disk 4 from Rod A to destination Rod C.

3,Recursively solve the puzzle of shifting the disk 1 , 2 , 3 from Rod B to Rod C.

Algorithm-Towers of Hanoi

- Our ultimate aim is to move disk n from the source to the destination and then place all the other $(n-1)$ disks on top of it.
- We can imagine applying the same idea recursively for the entire set of disks.

Algorithm-Towers of Hanoi

- The steps to follow are –
- Step 1 – Move $n-1$ disks from source to aux
- Step 2 – Move n th disk from source to dest
- Step 3 – Move $n-1$ disks from aux to dest

- START
Procedure Hanoi(disk, source, dest, aux)
IF disk == 1, THEN
 move disk from source to dest
ELSE
 Hanoi(disk - 1, source, aux, dest) //Step 1
 move disk from source to dest // Step 2
 Hanoi(disk - 1, aux, dest, source) // Step 3
END IF
END Procedure
END

Algorithm-Towers of Hanoi

```
void hanoi(int n, char from, char to, char via) {
    if(n == 1){
        printf("Move disk 1 from %c to %c\n", from, to);
    }
    else{
        hanoi(n-1, from, via, to);
        printf("Move disk %d from %c to %c\n", n, from, to);
        hanoi(n-1, via, to, from);
    }
}

int main() {
    int n = 3;
    char from = 'A';
    char to = 'B';
    char via = 'C';
    //calling hanoi() method
    hanoi(n, from, via, to);
}
```

Output

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

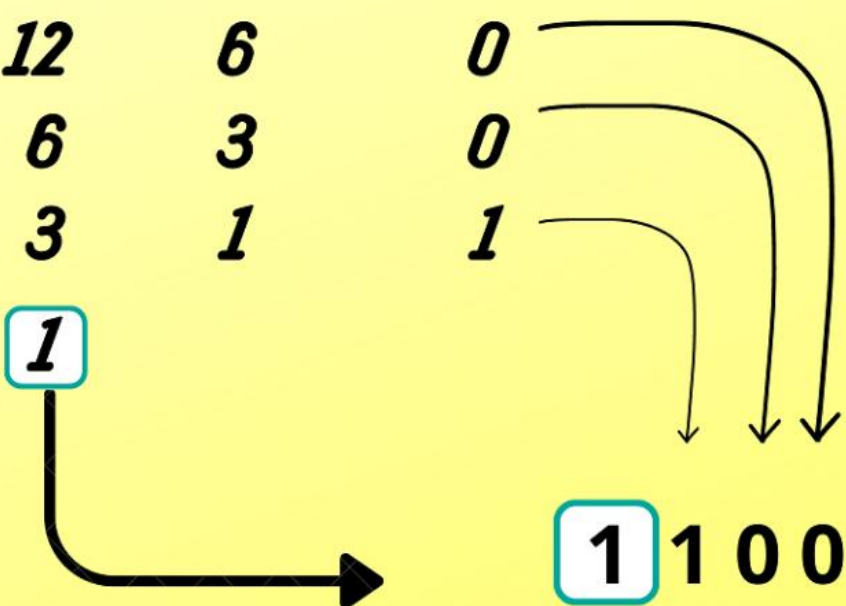
Write a recursive program to convert a decimal number to binary

Write a recursive program to convert a decimal number to binary

- Given a decimal number as input, we need to write a program to convert the given decimal number into an equivalent binary number.
- **Examples :**
- ***Input: $d = 7$***
Output: 111
Explanation: $2^0 + 2^1 + 2^2 = 1+2+4 = 7$.
- ***Input: $d = 10$***
Output: 1010
Explanation: $2^1 + 2^3 = 2+8 = 10$.

Write a recursive program to convert a decimal number to binary

-

n	$n/2$	$n\%2$	Output
12	6	0	
6	3	0	
3	1	1	
1			1 1 0 0

Write a recursive program to convert a decimal number to binary

```
#include <stdio.h>                                     // Driver code
// Decimal to binary conversion using recursion         int main()
int decToBin(int d)                                     {
{                                                       int d = 10;
    if (d == 0)                                         printf("%d", decToBin(d));
        return 0;                                     return 0;
    else                                                }
    return (d % 2 + 10 * decToBin(d /
2));
}
```

Explanation

```
if (d == 0)
```

```
    return 0;
```

- If the input d is zero, the binary representation is zero.
- **Recursive Case**
 - $d \% 2$ gives you the last binary digit (0 or 1).
 - $\text{decToBin}(d / 2)$ recursively converts the quotient.
 - $10 *$ shifts the previous result to the left in decimal representation, so the bits line up properly.

- **Example $d=5$**

- $5 \% 2 = 1$ (last bit)
- $5 / 2 = 2$
- Next call: $\text{decToBin}(2)$
- $2 \% 2 = 0$
- $2 / 2 = 1$
- Next call: $\text{decToBin}(1)$
- $1 \% 2 = 1$
- $1 / 2 = 0 \rightarrow$ Base case reached
- $\text{decToBin}(1) = 1$
- $\text{decToBin}(2) = 0 + 10 * 1 = 10$
- $\text{decToBin}(5) = 1 + 10 * 10 = 101$